

DEEPL - Lab 9

Self-Attention Mechanism

Nicolas Baskiotis - Stéphane Rivaud - Benjamin Piwowarski - Laure Soulier

2025-2026

Introduction (brief, cf. lecture)

In this lab, we will focus on *self-attention* mechanisms.

Unlike the previous lab, the goal of attention is not to build a fixed-size representation of a sequence $x_1^{(0)}, \dots, x_n^{(0)}$, but a sequence of *contextualized* representations $x_1^{(L)}, \dots, x_n^{(L)}$ where $x_i^{(L)}$ is the i -th contextualized input : each element of the input sequence computes its own representation, using an attention mechanism where the query is given by the element's representation. The process is repeated L times to obtain a better final representation.

Concretely, each element $x_i^{(l)}$ of the sequence after l layers is derived into three representations using three linear layers : a *query* representation $q_{\theta^{(l)}}(x_i^{(l)})$, a *key* representation $k_{\theta^{(l)}}(x_i^{(l)})$ and a *value* representation $v_{\theta^{(l)}}(x_i^{(l)})$. The query and key representations will allow us to compute the attention $\alpha_{ij}^{(l)}$ that element i pays to element j :

$$\log p(a_{ij}^{(l)}) = \text{constant} + \frac{1}{\sqrt{d}} q_{\theta^{(l)}}(x_i^{(l)}) \cdot k_{\theta^{(l)}}(x_j^{(l)})$$

where d is the dimension of the representations (the $d^{-1/2}$ allows us to always have the same gradient magnitudes regardless of the dimension).

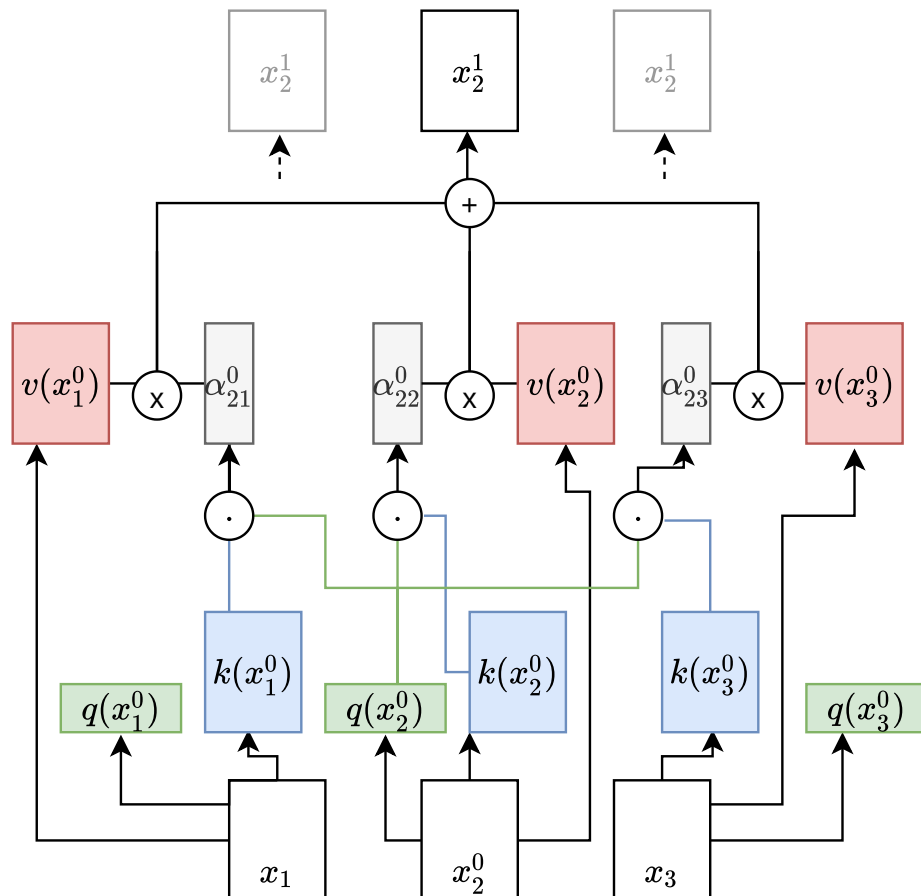
The representation $x_i^{(l+1)}$ of the element at the next layer is computed by taking the weighted average of the *value* of the elements from layer l weighted by the attention thus computed :

$$f_{\theta^{(l)}}(x_i^{(l)}; x_1^{(l)}, \dots, x_n^{(l)}) = \sum_{j=1}^n p(a_{ij}^{(l)}) v_{\theta^{(l)}}(x_j^{(l)}) \quad (1)$$

$f_{\theta^{(l)}}$ is a self-attention function where the query, key and value are linear functions of each input $x_i^{(l)}$. The functions are the same for all inputs but are distinct from one layer l to another (i.e. each module computing the query q , the key k or the value v has the same parameters for a given l).

We often introduce a final linear layer followed by a non-linearity (e.g., a ReLU function) to obtain the representation of $x_i^{(l+1)}$:

$$x_i^{(l+1)} = g_{\theta(v)}(f_{\theta(v)}(x_i^{(l)}; x_1^{(l)}, \dots, x_n^{(l)}))$$



Data and Model Preparation

Use the code from Lab 8 that you will extend to include models based on self-attention : as in Lab 8, you will use the IMDB data which contains 50,000 reviews from the Internet Movie Database (IMDb), and Glove word embeddings (see the provided code for the lab). The goal is to classify a review as being “positive” (pos) or “negative” (neg) ; the performance measure is the classification accuracy.

1 Base Model

Question 1

Implement a base model based on self-attention with $L = 3$. To classify, you will use the average as the final representation. Compare the performance with that of Lab 9 (in particular the mean module).

2 Residual Model

When the number of layers increases significantly, learning is greatly degraded. We use a residual architecture to address this problem :

$$x_i^{(l)} = g_{\theta} \left(\tilde{x}_i^{(l-1)} + f_{\theta}(\tilde{x}_i^{(l-1)}; \tilde{x}_1^{(l-1)}, \dots, \tilde{x}_n^{(l-1)}) \right)$$

Input normalization is often necessary to stabilize the network : Layer Normalization is used for this purpose to obtain the normalized inputs $\tilde{x}_i^{(l-1)}$.

Question 2

Implement the residual model and compare the results to the previous model.

3 Adding Positional Embeddings

The model described above has a significant problem : there is no notion of sequence taken into account, which is problematic for tasks such as sentiment detection (e.g., to properly account for negation, "I did not like" vs "I did not know that I would enjoy").

To address this problem, self-attention models use positional embeddings, i.e., we associate with each position i a vector pe_i whose h -th component is defined as follows (d is the dimension of the latent space) :

$$pe_{ih} = \begin{cases} \sin(i/10000^{h/d}) & \text{if } h \text{ is even} \\ \cos(i/10000^{(h-1)/d}) & \text{otherwise} \end{cases}$$

Question 3

In the provided code, you will find a `PositionalEncoding` class that allows you to add positional embeddings to sequences of representations ($\text{batch} \times \text{length} \times k$). You need to specify the maximum length of any batch at the start so that the vectors are pre-computed.

To understand what this function does, compute the dot product between pe_i and pe_j (for all i and j) in the form of a heatmap.

Modify the model from Exercise 1 by adding *positional embeddings* to the initial representation (i.e., replace the $x_i^{(0)}$ with $x_i^{(0)} + pe_i$).

It is also possible to learn a representation of positions rather than pre-computing them analytically. More recent models (like BERT) use a representation of relative positions rather than absolute ones.

4 Adding a CLS Token

Transformer-based models use another technique to compute a fixed-size representation for predicting the class, by introducing a special **CLS** token at the beginning of the sequence to classify, i.e., $x_1^{(0)} = x_{CLS}$. The embedding of this token is learned, and the contextualized representation $x_1^{(L)}$ is used to predict the class.

Question 4

Add the pseudo-token **CLS** and compare the performance with the previous model.

Links and References

- The original paper "Vaswani et al., Attention is All You Need, 2017"
- Transformers from scratch (August 2019) : a very well-made blog post on transformers.